

C++ & DSA Execution Plan

Divesh Shukla · March 31 → July 14, 2026

Status: Day 13 complete · 13 LeetCode problems solved · Ubuntu 24.04 environment ready

Checkpoint: May 31 · Full milestone: July 14

How to use this plan

This plan does not script your sessions.

Each unit gives you: what to build, what problems to solve, red flags for self-deception, exit criteria, and one conceptual question to answer cold. You decide how many days a unit takes. The Sunday audit tells you if you're on track. Exit criteria — not dates — tell you when you're done.

The three tags

☐ BLOCKING Missing this is breaking your current work. Fix it first. No workaround.	☐ PARALLEL Important but won't block today. Absorb it alongside blocking work. Don't halt DSA to learn it in isolation.	☐ DEFER Ignore until summer or Year 2. Time spent on it now comes out of blocking work.
--	--	--

Session structure (yours to own)

Block	Standard day (2.5 hrs)	Minimum viable day (2 hrs)
15 min	Retrieval warm-up — write yesterday's key concept from memory. Code one function cold, no reference.	Implement one thing from memory (30 min)
60 min	New concept — active reading. Every 10 min: close resource → write concept in own words → reopen and check. Never take notes while reading.	Solve one LeetCode problem (30 min)
60 min	Implementation from scratch. No reference. Valgrind on every memory unit. The errors you hit ARE the learning.	Write conceptual exit question answer (10 min)
15 min	One DSA problem — start it, don't necessarily finish it. Let it sit overnight. (Zeigarnik effect.)	<i>Floor is non-negotiable. Not even on exam nights.</i>

Sunday audit (20 min · every Sunday · no exceptions)

- What did I complete vs what was planned? (measure the gap)
- Was the gap from time, difficulty, or unclear concept? (diagnose the cause)
- What does next week look like given the gap? (adjust, don't ignore)

This is how you learn to plan. By July you will have done this ~15 times and will have a calibrated internal model of how long things actually take you.

Problem quota

Period	Floor (bad week)	Target (normal week)	Milestone
--------	------------------	----------------------	-----------

Now → May 14	1/day (7/week)	10/week	60 problems by May 14
May 15 → Jul 14	2/day (14/week)	20/week	100+ problems by Jul 14

Progress tracker

Update every Sunday audit. Status: *TODO* · *IN PROGRESS* · *DONE*

Unit	Title	Tag	Status	LC #
PHASE 2 — The Wall			TODO	
Pre-C++ B.3	Dynamic Memory malloc/free	BLOCKING	TODO	
C++ 1.1	C++ I/O & Types	BLOCKING	TODO	
C++ 1.2	Functions & Overloading	PARALLEL	TODO	
C++ 1.3	References	BLOCKING	TODO	
C++ 1.4	Memory Management new/delete	BLOCKING	TODO	
C++ 1.5	Exception Handling	PARALLEL	TODO	
DSA 0.2	Binary Search Mastery	BLOCKING	TODO	
DSA 0.3	Sorting & Prefix Sums	BLOCKING	TODO	
PHASE 3 — Convergence			TODO	
C++ 2.1	Classes & Encapsulation	BLOCKING	TODO	
C++ 2.2	Constructors & Destructors	BLOCKING	TODO	
C++ 2.3	Copy Semantics / Rule of Three	BLOCKING	TODO	
C++ 2.4	Operator Overloading	PARALLEL	TODO	
C++ 2.5	Inheritance & Polymorphism	PARALLEL	TODO	
DSA 1.1	Two Pointers & Sliding Window	BLOCKING	TODO	
DSA 1.2	String Manipulation	PARALLEL	TODO	
DSA 1.3	Hashing Techniques	BLOCKING	TODO	
DSA 1.4	Array Manipulation	PARALLEL	TODO	
DSA 2.1	Linked List Fundamentals	BLOCKING	TODO	

DSA 2.2	Advanced LL Patterns	PARALLEL	TODO	
PHASE 4 — Semester Close			TODO	
DSA 2.3	Stack Applications — Monotonic Stack	BLOCKING	TODO	
DSA 2.4	Queue & Deque Applications	PARALLEL	TODO	
C++ 3.2	STL Containers (min. subset)	PARALLEL	TODO	
TOTAL	LeetCode problems solved		13 / 100+	

PHASE 2 — The Wall

Mar 31 → Apr 20 (~21 days) · 2–2.5 hrs/day

Close out Pre-C++ (B.3 Dynamic Memory). Complete C++ Module 1 (I/O, References, new/delete) plus parallel units 1.2 and 1.5. DSA Layer 0 complete + Layer 1 start. This is the hardest phase — pointers and dynamic memory demand full attention. The parallel C++ units absorb inside existing sessions, never as separate study blocks.

PHASE EXIT — all must be true before advancing:

- DynArray struct: 100 pushes, valgrind zero errors zero leaks
- Binary search on answer (Koko-style) cold in under 6 minutes
- Pass-by-const-reference C++ function — correct signature from memory
- Overloaded function + custom exception — both work without reference
- 13 → 25+ LeetCode problems solved

❑ BLOCKING

Pre-C++ B.3 Dynamic Memory — malloc / free

~4 days · Manual heap allocation. Every program passes valgrind before you move on.

PRIMARY RESOURCE

- K.N. King — Chapter 17: Dynamic Memory Allocation (primary)
- CS:APP — Chapter 9.9: Dynamic Memory Allocation (the 'what malloc actually does' mental model)

WHAT TO BUILD

1. malloc/free single int — valgrind clean
2. malloc/free array of 10 ints — valgrind clean
3. calloc vs malloc — print uninitialized malloc value vs zeroed calloc, understand why
4. realloc to double array size — preserve existing elements, verify with print
5. Dangling pointer demo: free(p), then print p address only — never dereference
6. Double-free demo — run under valgrind, read the full error output
7. Dynamic 2D array: array of int* pointers, each pointing to a row of ints
8. Free 2D array correctly: loop to free each row, then free outer pointer array
9. DynArray struct { int* data; int size; int capacity; } with create(), push() doubling on full, get(), destroy() — ZERO valgrind errors, non-negotiable
10. Three separate programs, one each: use-after-free, double-free, memory leak — run each under valgrind, read and understand every line of output

PARALLEL Absorb C.1 const correctness and C.2 UB + sanitizers during this unit. Add const to every pointer parameter you write. Run -fsanitize=address on every program.

RED FLAGS — you are faking progress if:

- You run valgrind, see errors, and mark the task done anyway
- You can explain malloc/free but your DynArray still has a leak
- You skip the dangling pointer and double-free demos because they 'seem obvious'

EXIT CRITERIA — cannot move on until all are true:

- DynArray: 100 pushes, valgrind shows 'no errors detected, no leaks'
- Can write malloc/free for a 2D array from memory in under 8 minutes
- Can explain what free() does and doesn't do at the memory level without reference

CONCEPTUAL EXIT QUESTION

After calling free(p), what is the value of p? What happens if you dereference it? What happens if you free it again? Answer at the memory level — what is the CPU actually reading when you access freed memory?

Write the answer without reference before closing. If you can't — the concept isn't solid.

□ BLOCKING

C++ 1.1 C++ I/O & Types

~1 day · C → C++ idiom transition. Write idiomatic C++ without mentally translating from C.

PRIMARY RESOURCE

- A Tour of C++ — Chapter 1 (full, active reading)
- C++ Primer — Chapters 1, 2.1–2.2

WHAT TO BUILD

1. Hello world in C++ — cout, cin, namespace std
2. Read integers until EOF — cin >> n in while loop
3. auto type deduction — 5 examples, note what type each resolves to
4. Range-based for loop on a raw array, then on a vector
5. Convert your C my_strlen to C++ using std::string
6. nullptr vs NULL — demonstrate nullptr won't implicitly convert to int (compile error)

RED FLAGS — you are faking progress if:

- You still write printf and scanf out of habit in new C++ files
- You can't explain what auto actually does without saying 'it just figures it out'

EXIT CRITERIA — cannot move on until all are true:

- Write a C++ program using cin, cout, auto, range-for, std::string — no syntax errors, no reference
- Can explain why nullptr is type-safe and NULL is not

CONCEPTUAL EXIT QUESTION

What does auto actually do? Is it like var in JavaScript — does it give up type safety? What is the type of auto x = 3.14 and how does the compiler know?

Write the answer without reference before closing. If you can't — the concept isn't solid.

□ PARALLEL

C++ 1.2 Functions & Overloading

~1–2 days (absorb during 1.3) · Function overloading and default arguments — used in every C++ API and class you'll write.

PRIMARY RESOURCE

- C++ Primer — Chapter 6.4: Overloaded Functions, Chapter 6.5: Default Arguments
- A Tour of C++ — Chapter 1.3

WHAT TO BUILD

1. Overload `print()` for `int`, `double`, `string` — three versions, compiler picks correct one
2. Default argument: `void connect(string host, int port = 8080)` — call with and without port
3. Show ambiguous overload: what happens when two overloads could both match
4. `constexpr` function: `constexpr int square(int x)` — evaluated at compile time
5. inline function vs regular function — understand what inline suggests to the compiler

PARALLEL Do this alongside C++ 1.3 — one implementation exercise from this list per session while working on references.

RED FLAGS — you are faking progress if:

- You think function overloading means the function checks types at runtime
- You use default arguments but can't explain where they live in the declaration vs definition

EXIT CRITERIA — cannot move on until all are true:

- Can write three overloaded functions and explain how the compiler resolves which one to call
- Can write a `constexpr` function and use it in a `static_assert`

CONCEPTUAL EXIT QUESTION

How does the compiler decide which overloaded function to call? What is an ambiguous overload and when does it occur? What does `constexpr` mean on a function — where is it evaluated?

Write the answer without reference before closing. If you can't — the concept isn't solid.

❑ BLOCKING

C++ 1.3 References

~2 days · References are the default C++ parameter-passing idiom. Must be solid before writing any class methods.

PRIMARY RESOURCE

- C++ Primer — Chapter 2.3 (references), Chapter 6.2 (parameter passing)
- Effective C++ — Item 20: Prefer pass-by-reference-to-const to pass-by-value

WHAT TO BUILD

1. Reference declaration: `int& ref = x;` modify `ref`, observe `x` changes
2. Swap two integers using references — not pointers
3. Pass large struct by const reference — show it doesn't copy
4. Dangling reference: return reference to local variable — understand why this is UB
5. Three versions of same function: by value / by reference / by const reference — observe which modifies caller, which copies
6. Matrix addition using const reference parameters

7. Function that takes string& and reverses it in place

PARALLEL Absorb C++ 1.2 (Functions & Overloading) during this unit — one exercise from 1.2 per session.

RED FLAGS — you are faking progress if:

- You write int& ref; (uninitialized reference) and don't understand why it won't compile
- You think const int& and int const& are different
- You can't answer 'can a reference be null?' without hesitating

EXIT CRITERIA — cannot move on until all are true:

- Correct const-reference function signatures from memory, no reference
- Can name three things a reference cannot do that a pointer can
- Can identify a dangling reference in code and explain the UB

CONCEPTUAL EXIT QUESTION

What are three things a reference can't do that a pointer can? What is one guarantee a reference provides that a pointer doesn't? When should you choose a pointer over a reference in C++ — give the rule of thumb and a concrete example.

Write the answer without reference before closing. If you can't — the concept isn't solid.

❑ BLOCKING

C++ 1.4 Memory Management — new / delete

~2 days · new calls constructors, delete calls destructors. This is the conceptual bridge to RAII and smart pointers.

PRIMARY RESOURCE

- C++ Primer — Chapter 12.1 (new/delete section only — stop before smart pointers)
- Effective C++ — Item 16: Use the same form in corresponding uses of new and delete

WHAT TO BUILD

1. Allocate int with new, delete — valgrind clean
2. Allocate array with new[], delete[] — valgrind clean, note the [] must match
3. Class with print in constructor/destructor — demonstrate new calls ctor, delete calls dtor
4. Allocate 2D array with new — array of int* pointers to rows
5. Free 2D array correctly with nested delete[]
6. Convert your C DynArray struct to use new/delete instead of malloc/free
7. Mismatch demo: new[] then delete (without []) — run under valgrind, read the error

PARALLEL Absorb C++ 1.5 (Exception Handling basics) during this unit — do the last two implementation tasks from 1.5 in the same session.

RED FLAGS — you are faking progress if:

- You use new and delete[] together (or new[] and delete) and don't see why it's wrong

- You think delete frees the pointer variable — it frees the memory the pointer points to

EXIT CRITERIA — cannot move on until all are true:

- Can explain why new[] must pair with delete[] — trace the failure if mismatched
- Every new/delete exercise passes valgrind zero errors
- Can explain in one sentence why new is not just malloc with extra steps

CONCEPTUAL EXIT QUESTION

Why does new call constructors but malloc doesn't? Why does this matter for C++ objects specifically? What is the exact consequence of using delete on memory allocated with new[]?

Write the answer without reference before closing. If you can't — the concept isn't solid.

PARALLEL

C++ 1.5 Exception Handling Basics

~1–2 days (absorb during 1.4) · Exception safety is not optional in C++. Constructors can fail, destructors must not throw, noexcept matters for performance.

PRIMARY RESOURCE

- A Tour of C++ — Chapter 3.5: Error Handling
- C++ Primer — Chapter 18.1: Exception Handling
- Effective C++ — Item 29: Strive for exception-safe code

WHAT TO BUILD

1. throw and catch std::runtime_error — basic try/catch block
2. Custom exception class: derive from std::exception, override what()
3. Stack unwinding demo: local objects with print in destructor — observe they are destroyed during exception propagation
4. Catch by const reference (correct) vs catch by value (incorrect — object slicing)
5. noexcept on a function — understand what it promises and what happens if violated
6. Exception-safe function: if it throws, no resource leak — demonstrate with valgrind

PARALLEL Do this alongside C++ 1.4 — the last two exercises in 1.4's implementation list are from this unit.

RED FLAGS — you are faking progress if:

- You catch by value and wonder why the exception type info is wrong
- You think noexcept means the function can't throw — it means if it does, std::terminate is called

EXIT CRITERIA — cannot move on until all are true:

- Can write a try/catch block catching by const reference — no reference
- Can demonstrate stack unwinding with destructor prints
- Can explain what noexcept actually guarantees

CONCEPTUAL EXIT QUESTION

What is the difference between catching by value and catching by const reference? Why is catching by value incorrect when exception hierarchies are involved? What does noexcept actually promise — and what happens at runtime if the promise is broken?

Write the answer without reference before closing. If you can't — the concept isn't solid.

❑ BLOCKING

DSA 0.2 Binary Search Mastery

~Closing out (Day 12 implementations done) · Binary search is a pattern — search space reduction — not just an algorithm. Cold-executable in under 5 minutes.

PRIMARY RESOURCE

- CLRS — Chapter 2.3: Designing Algorithms (binary search proof of correctness)
- Implementations already in repo from Day 12 — move directly to problems

WHAT TO BUILD

1. If any Day 12 variant is shaky: reimplement it cold, timed — lower_bound, upper_bound, first/last occurrence, floor/ceiling
2. Binary search on answer template: define search space → define predicate → apply search

LEETCODE PROBLEMS

- LC 704 — Binary Search (must solve in under 3 min — baseline)
- LC 33 — Search in Rotated Sorted Array
- LC 153 — Find Minimum in Rotated Sorted Array
- LC 69 — Sqrt(x) via binary search
- LC 875 — Koko Eating Bananas ✓ done
- LC 1011 — Capacity To Ship Packages Within D Days
- LC 410 — Split Array Largest Sum (hard — attempt fully)
- LC 162 — Find Peak Element

For every problem: write invariant / approach before coding · write complexity after · if $O(n^2)$ exists find $O(n)$ first.

RED FLAGS — you are faking progress if:

- You solve each binary search problem by memorizing its specific code rather than deriving from the invariant
- You need more than 5 minutes for standard binary search from memory
- You can't identify that LC 410 is a binary-search-on-answer problem without being told

EXIT CRITERIA — cannot move on until all are true:

- Standard binary search (correct loop invariant) from memory in under 5 minutes cold
- Can apply binary search on answer to a new problem without being told to
- LC 410 attempted with a working approach

CONCEPTUAL EXIT QUESTION

What is the loop invariant in binary search? Why does $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$ instead of $(\text{left} + \text{right}) / 2$ matter? How do you decide whether to use $\text{left} < \text{right}$ or $\text{left} \leq \text{right}$ as the loop condition — what determines the choice?

Write the answer without reference before closing. If you can't — the concept isn't solid.

❑ BLOCKING

DSA 0.3 Sorting & Prefix Sums

~3 days · Sorting is already known. Prefix sums are the gap — every subarray problem in Layer 1 requires them.

PRIMARY RESOURCE

- CLRS — Chapter 8: Sorting in Linear Time (counting sort, radix sort — for the mental model of non-comparison sorting)
- Prefix sums: no single source — derive $\text{sum}(l, r) = \text{prefix}[r] - \text{prefix}[l-1]$ from first principles

WHAT TO BUILD

1. Prefix sum array from input
2. Range sum query: $\text{sum}(l, r) = \text{prefix}[r] - \text{prefix}[l-1]$
3. Edge case: $l = 0$ — handle cleanly without special casing
4. 2D prefix sum array — build and query
5. Difference array for range updates (complement of prefix sums)
6. Custom comparator for `sort()`: sort pairs by second element, then by first descending
7. Counting sort from scratch — verify it is $O(n+k)$ and understand when k makes it impractical

LEETCODE PROBLEMS

- LC 303 — Range Sum Query Immutable
- LC 304 — Range Sum Query 2D Immutable
- LC 560 — Subarray Sum Equals K (prefix sum + hash — key problem)
- LC 523 — Continuous Subarray Sum
- LC 56 — Merge Intervals (sort by start, then merge)
- LC 179 — Largest Number (custom comparator — the comparison is not obvious)

For every problem: write invariant / approach before coding · write complexity after · if $O(n^2)$ exists find $O(n)$ first.

RED FLAGS — you are faking progress if:

- You use $\text{prefix}[r] - \text{prefix}[l-1]$ without being able to derive why it works
- You handle $l=0$ with an if-statement instead of building the prefix array to avoid it
- You memorize the LC 179 comparator without understanding why the string concatenation comparison is correct

EXIT CRITERIA — cannot move on until all are true:

- Can derive prefix sum formula from first principles without reference
- Can solve LC 560 in under 15 minutes
- Can write a custom comparator for `sort()` without syntax errors

CONCEPTUAL EXIT QUESTION

Why does $\text{sum}(l, r) = \text{prefix}[r] - \text{prefix}[l-1]$ work? Derive it from the definition of $\text{prefix}[i]$. What is the edge case when $l = 0$ — what goes wrong with $\text{prefix}[l-1]$ and how do you handle it cleanly without an if-statement?

Write the answer without reference before closing. If you can't — the concept isn't solid.

PHASE 3 — Convergence

Apr 21 → May 14 (~24 days) · 2–3 hrs/day · exam protocol embedded

C++ Module 2: Classes → Constructors → Copy Semantics. DSA Layer 1 complete + Layer 2 start. Tracks begin feeding each other. Exam period runs inside this phase — the protocol below governs exam days, not a separate phase.

PHASE EXIT — all must be true before advancing:

- Rule of Three for any class with dynamic memory — zero valgrind errors from memory
- 50+ LeetCode problems solved by May 14
- Linked list reverse (iterative) in under 3 minutes cold
- Can identify DSA pattern family from problem statement without being told
- Operator overloading: stream operator and subscript operator — correct, no reference

EXAM PERIOD PROTOCOL (applies inside Phase 3 from approx. late April)

Evening before any exam: minimum viable day only — DSA problems, no new C++ theory.

Morning after exam: full session resumes immediately — no grace period.

Weekends with no paper the next day: full session, no exception.

❑ BLOCKING

C++ 2.1 Classes & Encapsulation

~2 days · Solidify empirical class knowledge. *const* member functions, *this* pointer, access discipline.

PRIMARY RESOURCE

- C++ Primer — Chapter 7: Classes (full chapter, active reading)
- Effective C++ — Item 3: Use *const* whenever possible

WHAT TO BUILD

1. Rectangle class — width, height, area(), perimeter(), all *const*-correct
2. BankAccount class — private balance, deposit(), withdraw() with validation
3. *this* pointer — method chaining: return **this* from setX(), observe chaining works
4. struct vs class — demonstrate the only difference is default access specifier
5. *const* member function vs non-*const*: int getX() *const* vs void setX(int) — both in same class
6. Static member variable — counter of how many instances exist
7. Private helper method — used internally, hidden from public interface
8. Complex number class — real, imag, add(), subtract(), magnitude(), print()

RED FLAGS — you are faking progress if:

- You add *const* to member functions only when the compiler forces you, not by default
- You can't explain what the *this* pointer is without looking it up
- You make data members public because 'it's easier'

EXIT CRITERIA — cannot move on until all are true:

- Class with *const* member functions, private members, static member variable — from memory, no reference
- Can explain what *const* at end of member function prevents — give a concrete example

- Can explain this and when returning `*this` is necessary

CONCEPTUAL EXIT QUESTION

What does `const` at the end of a member function declaration actually tell the compiler? What does the compiler prevent you from doing inside that function? Give a concrete example of a bug that `const` member functions catch at compile time.

Write the answer without reference before closing. If you can't — the concept isn't solid.

❑ BLOCKING

C++ 2.2 Constructors & Destructors

~3 days · Initialization lists, constructor order, `explicit` keyword. Getting initialization order wrong causes silent UB.

PRIMARY RESOURCE

- C++ Primer — Chapters 7.1, 7.5
- Effective C++ — Item 4 (initialize before use), Item 5 (know what functions C++ silently writes), Item 6 (explicitly disallow compiler-generated functions)

WHAT TO BUILD

1. Default constructor vs parameterized constructor in same class
2. Initialization list: `MyClass(): x(5) {}` vs body assignment — observe they produce different code
3. Initialization order: members initialize in declaration order, not initialization-list order — demonstrate with print
4. Destructor with print — observe destruction order is reverse of construction
5. Delegating constructor (C++11): one constructor calls another to avoid duplication
6. `explicit` keyword: prevent implicit conversion from `int` to your class — show the bug it prevents
7. RAII file handle: open in constructor, close in destructor — never leave resource open
8. `const` member initialization — must use initialization list, not body (demonstrate why body fails to compile)
9. Reference member initialization — same constraint, same reason
10. RAII lock guard: lock in constructor, unlock in destructor

RED FLAGS — you are faking progress if:

- You initialize members in the constructor body and assume the order is your initialization-list order
- You implement RAII by calling `close()` manually at the end of the function — that is not RAII
- You don't use `explicit` on single-argument constructors by default

EXIT CRITERIA — cannot move on until all are true:

- Can explain why `const` and reference members must use initialization list — not because the compiler says so, but why at the language level
- RAII file handle works: opens in constructor, closes in destructor even if exception thrown
- Destructor print demo: construction order then reverse destruction order, correct every time

CONCEPTUAL EXIT QUESTION

Why must const and reference members be in the initialization list and not the constructor body? What has already happened by the time the constructor body begins executing? If your initialization list is MyClass(): b(1), a(2) but a is declared before b — what is the actual initialization order and why?

Write the answer without reference before closing. If you can't — the concept isn't solid.

❑ BLOCKING

C++ 2.3 Copy Semantics — Rule of Three

~7 days — do not rush · The most important C++ unit. Deep copy, Rule of Three, copy-and-swap, self-assignment. Valgrind must be clean on everything.

PRIMARY RESOURCE

- C++ Primer — Chapters 13.1 and 13.2 (copy control)
- Effective C++ — Item 10 (operator= returns *this), Item 11 (handle self-assignment), Item 12 (copy all parts of an object)
- More Effective C++ — Item 17: consider copy-and-swap

WHAT TO BUILD

1. Shallow copy failure: class with int* data → default copy → two objects share pointer → first destructor frees → second destructor double-frees → valgrind catches it
2. Deep copy constructor: allocate new memory, copy value — valgrind clean
3. Copy assignment operator with self-assignment check: if (this == &other) return *this;
4. Copy-and-swap idiom: implement swap() as friend, implement operator= using it — this is exception-safe
5. Delete copy operations for intentionally non-copyable class (= delete)
6. Full Rule of Three for String class (char* buffer): constructor, destructor, copy constructor, copy assignment — ZERO valgrind errors
7. Full Rule of Three for DynArray class — same requirement
8. Demonstrate compiler-generated copy with a class that has a destructor — show the double-free that results

PARALLEL Absorb C++ 2.4 (Operator Overloading) after 2.3 is done but before 2.5. The stream operator and subscript operator are the two that matter most.

RED FLAGS — you are faking progress if:

- Your Rule of Three implementation passes valgrind on the happy path but you haven't tested self-assignment
- You implement copy assignment without a self-assignment check and think it will 'probably be fine'
- You claim to understand copy-and-swap but can't explain why it is exception-safe
- You rush past the shallow copy failure demo because you 'already get it'

EXIT CRITERIA — cannot move on until all are true:

- Rule of Three for any pointer-owning class — zero valgrind errors — from memory, no reference
- Self-assignment works correctly — demonstrated, not assumed
- String class: copy, assign to self, assign to other, destroy — all valgrind clean
- Can explain copy-and-swap and why it is exception-safe without reference

CONCEPTUAL EXIT QUESTION

What is the Rule of Three? Why does implementing a destructor imply you need a copy constructor and copy assignment operator? Trace through the exact memory failure: class A has `int*` data, you implement the destructor but use compiler-generated copy — what happens in memory when you do `A b = a;` and then both go out of scope?

Write the answer without reference before closing. If you can't — the concept isn't solid.

□ PARALLEL

C++ 2.4 Operator Overloading

~3 days (after 2.3 complete) · Custom operators appear in every ML library and DSA class you'll write. Stream and subscript operators are non-negotiable.

PRIMARY RESOURCE

- C++ Primer — Chapter 14: Overloaded Operations and Conversions
- Effective C++ — Item 23: Don't try to return a reference when you must return an object

WHAT TO BUILD

1. `operator+` for Complex class — returns new object (not `*this`)
2. `operator+=` for Complex — modifies and returns `*this`
3. `operator==` and `operator!=` — symmetric
4. `operator<` — enables sorting in STL containers
5. `operator<<` for ostream — must be non-member friend function
6. `operator>>` for istream — non-member friend
7. `operator[]` for array class — return reference (allows assignment through `[]`)
8. Prefix `++` vs postfix `++` — postfix takes dummy int parameter, prefix returns `*this`
9. Matrix class: `operator+`, `operator*`, `operator[]`, `operator<<`

RED FLAGS — you are faking progress if:

- You implement `operator<<` as a member function — it cannot be, because the left operand would be the object not the stream
- You return by value from `operator[]` when you need assignment through `[]` to work
- You implement `operator==` but forget `operator!=`

EXIT CRITERIA — cannot move on until all are true:

- Can implement `operator<<` as a non-member friend function — explain why it must be non-member
- `operator[]` returns reference so assignment through `[]` works
- Can explain the difference between returning by value and by reference from an operator

CONCEPTUAL EXIT QUESTION

Why must `operator<<` be a non-member function? What does friend do when applied to it? What is the rule for when an operator should return by value vs by reference — and what breaks if you get it wrong?

Write the answer without reference before closing. If you can't — the concept isn't solid.

□ PARALLEL

C++ 2.5 Inheritance & Polymorphism

~4–5 days · Virtual functions, virtual destructor rule, object slicing. Focus on correctness, not design patterns.

PRIMARY RESOURCE

- C++ Primer — Chapter 15: Object-Oriented Programming
- Effective C++ — Item 7 (declare destructors virtual in polymorphic base classes), Item 34, Item 36

WHAT TO BUILD

1. Shape base class — virtual area(), virtual perimeter()
2. Circle and Rectangle deriving from Shape — override both with override keyword
3. Polymorphism: Shape* array → call area() on each → correct derived version called
4. Virtual destructor necessity: Shape* s = new Circle(); delete s; — without virtual destructor valgrind shows leak, with it clean
5. Pure virtual function — abstract class that cannot be instantiated
6. Object slicing: pass Circle by value to function expecting Shape — demonstrate behavior loss
7. dynamic_cast: safe downcast from Shape* to Circle* — handle nullptr return
8. Animal hierarchy — Dog, Cat deriving from Animal — polymorphic speak()

RED FLAGS — you are faking progress if:

- You forget the virtual destructor and think 'it probably doesn't matter in practice'
- You use override but don't understand what it actually checks — it verifies you're genuinely overriding a virtual function
- You think object slicing is a rare edge case — it is a common source of bugs

EXIT CRITERIA — cannot move on until all are true:

- Virtual destructor demo: valgrind shows leak without it, clean with it
- Can explain object slicing — when it happens and how to prevent it
- Can explain vtable at a conceptual level — why virtual function calls have overhead

CONCEPTUAL EXIT QUESTION

What is object slicing? When does it happen? Why does passing a derived object by value to a function expecting a base class lose derived behavior? Separately: what exactly goes wrong at the memory level if the base class destructor is not virtual?

Write the answer without reference before closing. If you can't — the concept isn't solid.

□ BLOCKING

DSA 1.1 Two Pointers & Sliding Window

~6 days · Two pointers reduces $O(n^2)$ to $O(n)$ by maintaining an invariant. State the invariant before writing one line of code.

PRIMARY RESOURCE

- No single textbook — derive the invariant from first principles for every problem

- Key question before every problem: what invariant are my two pointers / window boundaries maintaining?

WHAT TO BUILD

1. Fixed sliding window template: sum of window of size k — implement and trace
2. Variable sliding window template: expand when window invalid, shrink when valid — implement general form
3. Two pointer template: left and right on sorted array, move based on comparison — implement general form

LEETCODE PROBLEMS

- LC 167 — Two Sum II (sorted) — warm-up, write invariant first
- LC 26 — Remove Duplicates from Sorted Array
- LC 283 — Move Zeroes
- LC 11 — Container With Most Water
- LC 15 — 3Sum (sort first, two pointers inside loop)
- LC 75 — Sort Colors (Dutch National Flag — three-way partition)
- LC 42 — Trapping Rain Water (two pointer)
- LC 643 — Maximum Average Subarray I (fixed window warm-up)
- LC 3 — Longest Substring Without Repeating Characters (variable window)
- LC 209 — Minimum Size Subarray Sum (variable window)
- LC 424 — Longest Repeating Character Replacement
- LC 76 — Minimum Window Substring (hard — attempt fully, do not skip)

For every problem: write invariant / approach before coding · write complexity after · if $O(n^2)$ exists find $O(n)$ first.

PARALLEL Absorb DSA 1.2 (String Manipulation) parallel to this unit — string problems share patterns with sliding window.

RED FLAGS — you are faking progress if:

- You solve sliding window problems by adjusting code until it works rather than deriving from the invariant
- You can't state the window invariant for LC 3 without looking at your solution
- You skip LC 76 because it looks hard

EXIT CRITERIA — cannot move on until all are true:

- Can state the window invariant for any sliding window problem before coding
- LC 76 solved — working $O(n)$ approach
- Can explain: fixed vs variable window, when to expand, when to shrink

CONCEPTUAL EXIT QUESTION

What is the difference between a fixed and a variable sliding window? In a variable window, when do you expand and when do you shrink? Write the general pseudocode template for a variable window. What is the invariant in LC 76 (Minimum Window Substring)?

Write the answer without reference before closing. If you can't — the concept isn't solid.

□ PARALLEL

DSA 1.2 String Manipulation

~3–4 days (absorb during 1.1) · String problems are unavoidable in interviews. Most patterns (sliding window on strings, two pointers on chars) overlap with DSA 1.1.

PRIMARY RESOURCE

- C++ Primer — Chapter 9.5: Additional string Operations
- Problems are the primary resource here — patterns emerge from solving

WHAT TO BUILD

1. Anagram check: sort both strings, compare — then $O(n)$ version with frequency array
2. Palindrome check: two pointer from both ends
3. Reverse words in a string in place — multiple passes
4. String compression: 'aabccc' → 'a2b1c3' — implement without extra library

LEETCODE PROBLEMS

- LC 125 — Valid Palindrome
- LC 242 — Valid Anagram
- LC 49 — Group Anagrams (unordered_map + sorted string as key)
- LC 151 — Reverse Words in a String
- LC 8 — String to Integer (atoi) — edge cases matter
- LC 28 — Find the Index of the First Occurrence in a String
- LC 443 — String Compression

For every problem: write invariant / approach before coding · write complexity after · if $O(n^2)$ exists find $O(n)$ first.

PARALLEL Do this alongside DSA 1.1 — interleave: one sliding window problem, one string problem, per session.

RED FLAGS — you are faking progress if:

- You use `std::sort` on a string for anagram checks without knowing the complexity
- You handle palindrome problems with a new reversed string instead of two pointers

EXIT CRITERIA — cannot move on until all are true:

- Can solve anagram check two ways: sort $O(n \log n)$ and frequency array $O(n)$ — know when each applies
- Can solve LC 49 (Group Anagrams) using sorted string as unordered_map key

CONCEPTUAL EXIT QUESTION

What is the time complexity of sorting a string for anagram detection vs using a frequency array? When does the frequency array approach fail? What is the key insight in Group Anagrams — why does using the sorted string as a hash key work?

Write the answer without reference before closing. If you can't — the concept isn't solid.

□ BLOCKING

DSA 1.3 Hashing Techniques

~6 days · Complement pattern and prefix sum + hash are the most tested patterns at any interview level.

PRIMARY RESOURCE

- CLRS — Chapter 11: Hash Tables (collision handling, load factor — conceptual only)
- C++ Primer — Chapter 11.3: unordered containers (for implementation)

WHAT TO BUILD

1. unordered_map: insert, find, erase, iterate — 5 distinct exercises
2. unordered_set: O(1) membership test — 3 exercises
3. Complement pattern from scratch: store seen values, look up complement — derive why it's O(n) not O(n²)
4. Prefix sum + hash pattern: for subarray sum problems — implement the template

LEETCODE PROBLEMS

- LC 1 — Two Sum (complement pattern — implement and prove O(n))
- LC 217 — Contains Duplicate
- LC 128 — Longest Consecutive Sequence
- LC 560 — Subarray Sum Equals K (prefix + hash — revisit if done earlier)
- LC 523 — Continuous Subarray Sum
- LC 974 — Subarray Sums Divisible by K
- LC 36 — Valid Sudoku
- LC 380 — Insert Delete GetRandom O(1) (design — unordered_map + vector)
- LC 146 — LRU Cache (hash + doubly linked list — this is the exit criterion)

For every problem: write invariant / approach before coding · write complexity after · if O(n²) exists find O(n) first.

PARALLEL Pick up DSA 1.4 (Array Manipulation) after 1.1 and 1.3 are solid — parallel to Layer 2 start.

RED FLAGS — you are faking progress if:

- You can implement Two Sum but can't prove why it's O(n) — 'because hash map is O(1)' is not a proof
- You don't attempt LRU Cache because it looks like a hard problem — it is, that's the point
- You can't explain what worst-case O(n) for unordered_map means or when it happens

EXIT CRITERIA — cannot move on until all are true:

- LRU Cache (LC 146) solved — this is the exit criterion, not the problem count
- Can prove complement pattern is O(n) — not just state it
- Can explain when and why unordered_map degrades to O(n) per operation

CONCEPTUAL EXIT QUESTION

Why does the complement pattern work for Two Sum? Prove the hash map approach is O(n) and not O(n²) — what assumption about hash map operations does this proof require? What is the worst case for unordered_map and what causes it?

Write the answer without reference before closing. If you can't — the concept isn't solid.

□ PARALLEL

DSA 1.4 Array Manipulation Patterns

~4 days (after 1.1 + 1.3 solid) · Kadane's, Product Except Self, Rotate Image, Jump Game — high-frequency patterns that require derivation, not memorization.

PRIMARY RESOURCE

- CLRS — for Kadane's: think of it as a DP recurrence: $dp[i] = \max(arr[i], dp[i-1] + arr[i])$
- Problems are the primary resource — derive each algorithm before looking up the standard solution

WHAT TO BUILD

1. Kadane's algorithm: derive from the subproblem 'max subarray sum ending at index i' — don't memorize, derive
2. Product of array except self: left pass then right pass — no division, $O(1)$ extra space
3. Rotate array left by k: three-reversal trick — implement and prove why it works

LEETCODE PROBLEMS

- LC 53 — Maximum Subarray (Kadane's — derive the algorithm, don't look it up first)
- LC 238 — Product of Array Except Self (no division allowed)
- LC 189 — Rotate Array
- LC 121 — Best Time to Buy and Sell Stock
- LC 55 — Jump Game
- LC 56 — Merge Intervals (revisit from 0.3 — now apply cleanly)
- LC 48 — Rotate Image (in-place matrix rotation)
- LC 54 — Spiral Matrix

For every problem: write invariant / approach before coding · write complexity after · if $O(n^2)$ exists find $O(n)$ first.

RED FLAGS — you are faking progress if:

- You look up Kadane's algorithm before attempting to derive it — this defeats the purpose
- You solve Rotate Image by copying to a new matrix instead of finding the in-place pattern

EXIT CRITERIA — cannot move on until all are true:

- Can derive Kadane's recurrence from first principles and explain why the answer is $\max(dp[0] \dots dp[n-1])$ not just $dp[n-1]$
- Can solve Product of Array Except Self in $O(n)$ time $O(1)$ extra space
- Can solve LC 55 (Jump Game) and explain the greedy invariant

CONCEPTUAL EXIT QUESTION

What is the recurrence for Kadane's algorithm? Define $dp[i]$ as the maximum subarray sum ending at index i. Write the recurrence. Why is the answer max over all $dp[i]$ and not $dp[n-1]$? What is the greedy insight in Jump Game?

Write the answer without reference before closing. If you can't — the concept isn't solid.

□ BLOCKING

DSA 2.1 Linked List Fundamentals

~4 days · Problem-solving speed on linked list patterns. You built one — now train implementation under time pressure.

PRIMARY RESOURCE

- Prior implementation is your baseline — focus is problem-solving speed, not re-learning
- Key technique: dummy node — use it on every problem involving head manipulation until it is automatic

WHAT TO BUILD

1. Iterative linked list reverse from memory — time yourself, target under 3 minutes
2. Recursive linked list reverse from memory
3. Dummy node pattern: implement merge two sorted lists using a dummy head
4. Floyd's cycle detection with correct null-safety on the double-step (fix the hasCycle null check bug from your original code)
5. Cycle start detection: slow from head, fast from meeting point — implement and verify on a constructed cycle

LEETCODE PROBLEMS

- LC 206 — Reverse Linked List (iterative AND recursive — both)
- LC 21 — Merge Two Sorted Lists (dummy node)
- LC 19 — Remove Nth Node From End
- LC 876 — Middle of Linked List (slow/fast pointer)
- LC 141 — Linked List Cycle (Floyd's — correct null-safety)
- LC 142 — Linked List Cycle II (find cycle start)
- LC 234 — Palindrome Linked List
- LC 143 — Reorder List
- LC 23 — Merge K Sorted Lists (hard — uses priority_queue)

For every problem: write invariant / approach before coding · write complexity after · if $O(n^2)$ exists find $O(n)$ first.

PARALLEL Absorb DSA 2.2 (Advanced LL Patterns) after 2.1 is solid.

RED FLAGS — you are faking progress if:

- Iterative reverse takes you more than 5 minutes — this means it's not automatic yet
- Your Floyd's implementation has the same null-safety bug as your original hasCycle code
- You use the dummy node only when reminded, not as a default reflex

EXIT CRITERIA — cannot move on until all are true:

- Iterative reverse from memory in under 3 minutes — timed, cold
- LC 141 and LC 142 both solved with correct null-safety on the double-step
- Dummy node pattern used without thinking on head-manipulation problems

CONCEPTUAL EXIT QUESTION

In Floyd's cycle detection, why do fast and slow always meet inside the cycle? Why does starting slow from head and fast from meeting point find the cycle start? Prove both — specifically prove the distance from head to cycle start equals distance from meeting point to cycle start.

Write the answer without reference before closing. If you can't — the concept isn't solid.

□ PARALLEL

DSA 2.2 Advanced Linked List Patterns

~3 days (after 2.1 solid) · Floyd's full mathematical treatment, merge K sorted, reorder list — the hard problems that require multiple simultaneous pointer invariants.

PRIMARY RESOURCE

- Problems are the primary resource — the patterns are extensions of 2.1 techniques
- Floyd's proof: work through the algebra — let distance head-to-cycle-start = a, cycle-start-to-meeting = b, remaining cycle = c

WHAT TO BUILD

1. Floyd's proof: implement with a constructed cycle where you know a, b, c — verify meeting point math
2. Reverse a sublist (from position m to n) in one pass — requires precise pointer tracking
3. Flatten a multilevel doubly linked list

LEETCODE PROBLEMS

- LC 24 — Swap Nodes in Pairs
- LC 25 — Reverse Nodes in k-Group (hard)
- LC 92 — Reverse Linked List II (sublist reversal)
- LC 148 — Sort List (merge sort on linked list — medium/hard)
- LC 430 — Flatten a Multilevel Doubly Linked List

For every problem: write invariant / approach before coding · write complexity after · if $O(n^2)$ exists find $O(n)$ first.

RED FLAGS — you are faking progress if:

- You attempt LC 25 before LC 24 — do them in order
- You solve sublist reversal by converting to array — that defeats the pointer manipulation practice

EXIT CRITERIA — cannot move on until all are true:

- Can solve LC 92 (Reverse Linked List II) in one pass from memory
- Can work through the Floyd's proof algebra without reference

CONCEPTUAL EXIT QUESTION

In Floyd's algorithm: let a = distance from head to cycle start, b = distance from cycle start to meeting point, c = remaining cycle length. Show algebraically why slow from head and fast from meeting point meet at the cycle start.

Write the answer without reference before closing. If you can't — the concept isn't solid.

PHASE 4 — Semester Close

May 1 → May 14 · 1.5 hrs min on exam days · 2.5 hrs otherwise

Exam period peaks here. Minimum viable day protocol mandatory on exam nights. DSA problems are your exam-day activity — low setup, no environment needed, keeps the streak alive. No new C++ theory on any night before a paper.

PHASE EXIT — all must be true before advancing:

- All Phase 2 and Phase 3 blocking units complete
- 60+ LeetCode problems solved
- STL minimum subset usable in problems without reference
- Exam period protocol held every exam night — never dropped below minimum viable day

□ BLOCKING

DSA 2.3 Stack Applications — Monotonic Stack

~4–5 days · Monotonic stack solves next-greater/smaller in $O(n)$. Must be pattern-fluent — derivable, not memorized.

PRIMARY RESOURCE

- No single source — derive the invariant: the stack maintains sorted order. When you pop, you've found that element's answer.
- Key insight: the stack never stores the answer — it stores candidates waiting for their answer.

WHAT TO BUILD

1. Valid parentheses from scratch — warm-up on stack mechanics
2. Min Stack: push, pop, top, getMin — all $O(1)$
3. Monotonic decreasing stack template: given array, find next greater element for each index — implement general template

LEETCODE PROBLEMS

- LC 20 — Valid Parentheses (warm-up)
- LC 155 — Min Stack
- LC 739 — Daily Temperatures (entry point for monotonic stack)
- LC 496 — Next Greater Element I
- LC 503 — Next Greater Element II (circular array)
- LC 84 — Largest Rectangle in Histogram (hard — the exit criterion for this unit)
- LC 42 — Trapping Rain Water (stack approach — compare with two-pointer approach from 1.1)
- LC 394 — Decode String

For every problem: write invariant / approach before coding · write complexity after · if $O(n^2)$ exists find $O(n)$ first.

RED FLAGS — you are faking progress if:

- You solve Daily Temperatures by memorizing the pattern rather than deriving from the invariant
- You skip LC 84 because it looks hard — it is the exit criterion, there is no skipping it
- You can't explain what 'the stack maintains sorted order' means concretely

EXIT CRITERIA — cannot move on until all are true:

- LC 84 (Largest Rectangle in Histogram) solved — this is the exit criterion

- Can explain monotonic stack invariant: what is maintained, what does a pop event mean
- Can identify from a problem statement whether monotonic stack applies

CONCEPTUAL EXIT QUESTION

Explain the monotonic stack invariant in Largest Rectangle in Histogram. When you pop a bar, what have you determined about that bar? Why is the width calculation $i - \text{stack.top()} - 1$ correct? Trace through bars [2, 1, 5, 6, 2] manually.

Write the answer without reference before closing. If you can't — the concept isn't solid.

PARALLEL

DSA 2.4 Queue & Deque Applications

~3 days · Sliding window maximum via monotonic deque is $O(n)$ and directly prepares BFS thinking for Layer 3.

PRIMARY RESOURCE

- Key insight for monotonic deque: remove from front when window expires, remove from back when new element dominates

WHAT TO BUILD

1. Queue using two stacks — understand amortized $O(1)$
2. Circular queue with fixed-size ring buffer
3. Monotonic deque template: deque that maintains max of current window

LEETCODE PROBLEMS

- LC 232 — Implement Queue using Stacks
- LC 225 — Implement Stack using Queues
- LC 622 — Design Circular Queue
- LC 239 — Sliding Window Maximum (monotonic deque — $O(n)$)
- LC 862 — Shortest Subarray with Sum at Least K (deque + prefix sum)

For every problem: write invariant / approach before coding · write complexity after · if $O(n^2)$ exists find $O(n)$ first.

RED FLAGS — you are faking progress if:

- You solve Sliding Window Maximum with a sorted structure ($O(n \log n)$) instead of monotonic deque ($O(n)$)
- You can't explain why you remove from the back of the deque — just that 'the algorithm says to'

EXIT CRITERIA — cannot move on until all are true:

- LC 239 solved and deque invariant explained cold
- Can explain: remove from front = expired window element, remove from back = dominated by new element

CONCEPTUAL EXIT QUESTION

In Sliding Window Maximum: what invariant does the deque maintain? Why do you remove from the front AND the back — what does each removal represent? Trace [3, 1, 3, 4, 2] with $k=3$ step by step.

Write the answer without reference before closing. If you can't — the concept isn't solid.

□ PARALLEL

C++ 3.2 STL Containers — Minimum Viable Subset

~2–3 days · `vector`, `unordered_map`, `unordered_set`, `stack`, `priority_queue` — enough to solve DSA problems in C++ without reference.

PRIMARY RESOURCE

- C++ Primer — Chapter 9 (vector), Chapter 11 (unordered containers)
- cppreference.com — API reference only while solving problems, not for learning

WHAT TO BUILD

1. `vector`: `push_back`, `insert`, `erase`, `iterate` — trigger iterator invalidation deliberately, observe crash
2. Correct `vector` `erase` in loop: use returned iterator, not the original
3. `unordered_map`: `insert` with `[]`, `insert` with `emplace`, `find`, `erase`, `iterate`
4. `unordered_set`: $O(1)$ membership test — 3 exercises with real use cases
5. `stack` adapter: `push`, `pop`, `top` — implement valid parentheses using it
6. `priority_queue` max-heap: push 10 elements, pop in order — verify heap property
7. `priority_queue` min-heap: correct comparator syntax `greater<int>`
8. `emplace_back` vs `push_back`: demonstrate the difference with a class that prints in constructor

RED FLAGS — you are faking progress if:

- You use `at()` vs `[]` without knowing the difference (`at()` throws, `[]` is UB on out-of-bounds)
- You write `priority_queue<int, vector<int>, less<int>>` for a max-heap — that's the default, you don't need it
- You erase from a vector inside a range-for loop

EXIT CRITERIA — cannot move on until all are true:

- All five containers usable in a DSA problem without consulting reference
- Can explain iterator invalidation: when it happens with vector and why at the memory level
- Can construct min-heap `priority_queue` with correct comparator syntax from memory

CONCEPTUAL EXIT QUESTION

What is iterator invalidation? When does it happen with vector — specifically what internal operation causes it? Write a code example that triggers it. How do you correctly erase elements from a vector while iterating?

Write the answer without reference before closing. If you can't — the concept isn't solid.

Flexibility rules

A plan without a feedback loop is a document. These rules are what keep it alive.

Rule 1 — Minimum viable day (non-negotiable floor)

On any bad day — exam tomorrow, sick, exhausted — exactly two things: implement one thing from memory (30 min), solve one LeetCode problem (30 min). Nothing else required. Nothing else excused either. This floor keeps your coding muscle and problem-solving streak alive through any disruption.

Rule 2 — Exam period protocol

Evening before any exam: minimum viable day only. DSA problems only — no new C++ theory. Morning after the exam: full session resumes, no grace period. Weekends with no paper the next day: full session, no exception. Exam periods become full coding dead zones when there is no explicit protocol. This is the protocol.

Rule 3 — Phase exit criteria are requirements, dates are targets

If April 20 arrives and Copy Semantics isn't solid — Phase 3 does not start. You extend Phase 2 and let summer absorb the slippage. The dates in this plan are targets. The exit criteria are requirements. A date without a capability criterion is a wish, not a plan.

Rule 4 — Sunday audit, every Sunday, no skipping

Three questions: what did I complete vs planned? Was the gap from time, difficulty, or unclear concept? What does next week look like given the gap? This is not optional maintenance. It is the mechanism by which you learn to plan. By July you will have done it ~15 times. By then your estimates will be calibrated, not guesses.

Rule 5 — Copy Semantics gets 7 days. Budget 7.

Every student believes they will finish Unit 2.3 in 3–4 days. It takes 7. If you finish in 5, you have 2 extra days — pull forward. If you need 9, the summer buffer absorbs it. Do not rush Rule of Three. A shaky 2.3 creates bugs in every C++ class you write afterward.

Rule 6 — Never drop the DSA streak on heavy C++ days

On days deep in copy semantics, new/delete, or constructor order — the temptation is to skip DSA. Don't. Minimum: 1 LeetCode problem, 15 minutes. Interleaving is what builds pattern transfer, not just pattern knowledge. The streak is what builds automaticity.

Rule 7 — GitHub commit every day

`git add .` && `git commit -m 'Unit X: what I built'` && `git push`. A commit is evidence. It makes progress visible and creates an honest record of what you actually built. If you didn't commit, the work doesn't count toward the plan.

Learning science embedded in this plan

Every structural decision is grounded in research. Understand why the structure exists and you can apply these principles to every plan you build after this one.

Retrieval practice (Roediger & Karpicke, 2006 — Science)

Every session opens with a retrieval warm-up: write yesterday's concept from memory before touching any material. This produces ~50% better long-term retention than re-reading. It is the highest-ROI 15 minutes in any session. The exit question at the end of each unit is the same mechanism — production, not consumption.

Spaced repetition (Ebbinghaus 1885; Cepeda et al. 2006)

The Sunday audit creates a 7-day review cycle. The daily retrieval warm-up creates a 1-day spacing. These two intervals sit in the optimal range for long-term retention. The plan doesn't add explicit flashcard review because the coding warm-up is more active than card review — it produces, it doesn't recognize.

Interleaving (Kornell & Bjork, 2008 — Journal of Experimental Psychology)

Running C++ implementation and DSA problems in the same session mixes two domains. Blocked practice (all C++ for a week, all DSA the next) feels faster but produces less transfer. Interleaving feels harder. That difficulty is the mechanism — it trains pattern recognition in novel contexts, which is exactly your end goal.

Generation effect (Slamecka & Graf, 1978)

Writing code from scratch produces deeper encoding than reading or modifying existing code. Every implementation task says 'from scratch, no reference.' The errors you generate during implementation are not obstacles to learning — they are the learning. This is why the plan has no 'follow along' exercises.

Desirable difficulty (Bjork, 1994)

The plan is designed to feel hard. Sessions that feel easy mean you're re-reading or solving below your level. The exit criteria are set at the edge of your current ability, not safely within it. Tier 2 problems in the original triage exist specifically to create difficulty. You will not solve all of them. That is by design.

Zeigarnik effect (Zeigarnik, 1927)

Every session ends by starting a problem without finishing it. Incomplete tasks stay active in working memory and receive continued background processing overnight. You will think about the problem during dinner, while walking, before sleep. That is not distraction — that is encoding. The plan uses this deliberately.